

Final Project: Physics-Informed Neural Networks for Solving Differential Equations

Alex Zhu

April 2026

Github with code linked here:

<https://github.com/threeierseries/Numerical-Analysis-Final-Project>

Note on repeated runs. The earlier problem sections show representative standalone runs for each method. Later comparison sections, such as Problem 1.4, Problem 2.4, and Problem 3(a), use separate final full runs so that the methods being compared are produced from the same execution workflow. Because the PINN methods involve random initialization, random collocation-point sampling, and nonconvex neural-network optimization, repeated runs can produce slightly different final losses, errors, and wall-clock times. Therefore, small differences between the standalone results in Problems 1.2, 1.3, 2.2, and 2.3 and the comparison results in Problems 1.4, 2.4, and 3(a) are expected. In particular, the values in Problem 1.3 and Problem 1.4 may differ because they come from different training runs of the same FDM-PINN method.

Problem 1.1

Part A

Using Forward Euler. Let

$$f(t, u) = -5u + 5\cos(t) - \sin(t).$$

With step size

$$h = 0.01,$$

define the grid points

$$t_n = nh, \quad n = 0, 1, \dots, N,$$

where

$$N = \frac{5 - 0}{0.01} = 500.$$

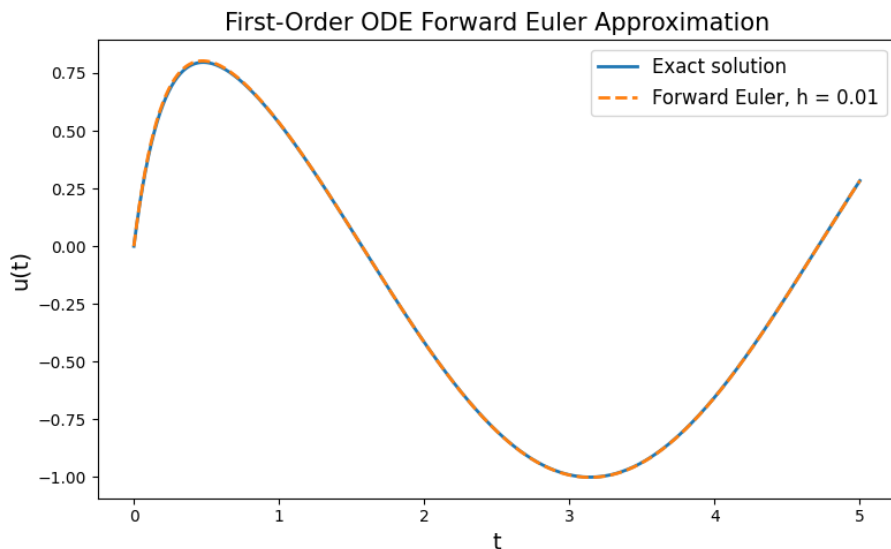
Forward Euler gives

$$w_{n+1} = w_n + hf(t_n, w_n),$$

so in this problem,

$$w_{n+1} = w_n + h(-5w_n + 5\cos(t_n) - \sin(t_n)), \quad w_0 = 0.$$

The numerical solution is plotted together with the exact solution below.



From the plot, the Forward Euler approximation follows the exact solution very closely on the interval $[0, 5]$. The two curves are almost indistinguishable, indicating that the step size $h = 0.01$ is small enough to give a visually accurate approximation. The numerical solution captures the main behavior of the exact solution. There is no visible numerical instability or large drift over the interval.

Part B

We now use the fourth-order Runge–Kutta method. We use the classical RK4 method from the textbook. For the general initial value problem

$$y'(t) = f(t, y(t)), \quad y(t_0) = y_0,$$

the RK4 method is given by

$$\begin{aligned} k_1 &= f(t_n, w_n), \\ k_2 &= f\left(t_n + \frac{h}{2}, w_n + \frac{h}{2}k_1\right), \\ k_3 &= f\left(t_n + \frac{h}{2}, w_n + \frac{h}{2}k_2\right), \end{aligned}$$

$$k_4 = f(t_n + h, w_n + hk_3),$$

and

$$w_{n+1} = w_n + \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4), \quad w_0 = y_0.$$

For this problem,

$$f(t, u) = -5u + 5 \cos(t) - \sin(t),$$

with

$$u(0) = 0.$$

Using step size

$$h = 0.01,$$

we have grid points

$$t_n = nh, \quad n = 0, 1, \dots, N,$$

where

$$N = \frac{5 - 0}{0.01} = 500.$$

Therefore, for each step,

$$k_1 = -5w_n + 5 \cos(t_n) - \sin(t_n),$$

$$k_2 = -5 \left(w_n + \frac{h}{2} k_1 \right) + 5 \cos \left(t_n + \frac{h}{2} \right) - \sin \left(t_n + \frac{h}{2} \right),$$

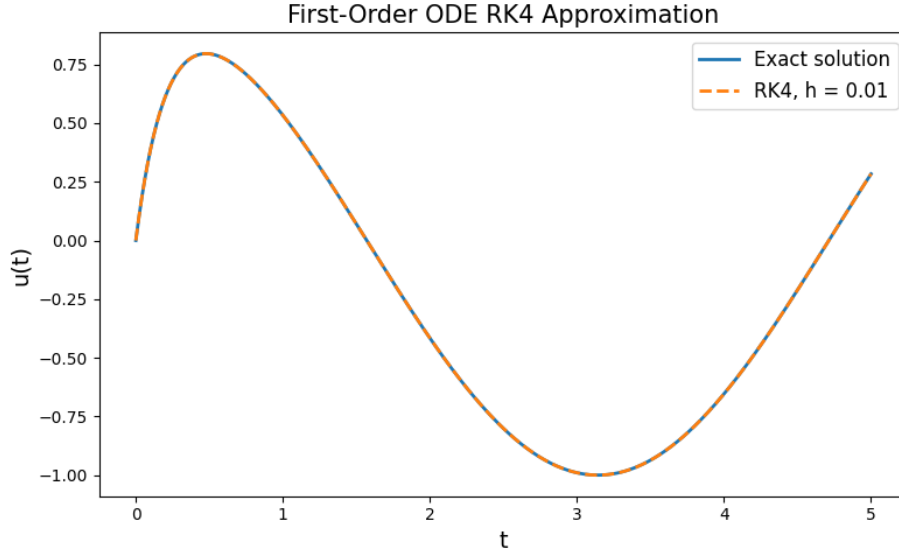
$$k_3 = -5 \left(w_n + \frac{h}{2} k_2 \right) + 5 \cos \left(t_n + \frac{h}{2} \right) - \sin \left(t_n + \frac{h}{2} \right),$$

$$k_4 = -5(w_n + hk_3) + 5 \cos(t_n + h) - \sin(t_n + h).$$

Then

$$w_{n+1} = w_n + \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4), \quad w_0 = 0.$$

The numerical solution is plotted together with the exact solution below.



From the plot, the RK4 approximation agrees very closely with the exact solution on the interval $[0, 5]$. The two curves are visually almost identical, which is expected because RK4 is a fourth-order method and the step size $h = 0.01$ is small. Compared with Forward Euler, RK4 should give a much smaller global error for the same step size.

Part C

For each step size h , we compute the global error

$$E_h = \max_{0 \leq n \leq N} |w_n - u(t_n)|,$$

where

$$t_n = nh, \quad N = \frac{5}{h}.$$

To estimate the observed convergence order, we know the error behaves like

$$E_h \approx Ch^p.$$

Using two step sizes h_1 and h_2 , we have

$$E_{h_1} \approx Ch_1^p, \quad E_{h_2} \approx Ch_2^p.$$

Dividing gives

$$\frac{E_{h_1}}{E_{h_2}} \approx \left(\frac{h_1}{h_2} \right)^p.$$

Taking logarithms and solving for p , we obtain

$$p \approx \frac{\log(E_{h_1}/E_{h_2})}{\log(h_1/h_2)}.$$

The first row has no observed order because the order formula requires two step sizes and two corresponding errors. Therefore, the order shown in the $h = 0.005$ row compares $h = 0.01$ with $h = 0.005$, and the order shown in the $h = 0.001$ row compares $h = 0.005$ with $h = 0.001$.

Part 1.1(c): Global Errors and Observed Convergence Orders				
h	Euler Error	Euler Order	RK4 Error	RK4 Order
0.01000	1.002978e-02	--	2.164679e-08	--
0.00500	4.964941e-03	1.0144	1.325852e-09	4.0292
0.00100	9.850170e-04	1.0050	2.086664e-12	4.0102

The Forward Euler method has observed order approximately 1, which agrees with the expected first-order convergence. The classical RK4 method has observed order approximately 4, which agrees with the expected fourth-order convergence. Also, RK4 gives a much smaller error than Forward Euler for the same step size.

Problem 1.2

Part A

Following the structure provided, for the automatic differentiation PINN, the neural network $u_\theta(t)$ is trained to satisfy

$$\frac{du}{dt} = -5u + 5\cos(t) - \sin(t), \quad u(0) = 0.$$

Equivalently, the differential equation residual is

$$R_\theta(t) = \frac{du_\theta}{dt}(t) + 5u_\theta(t) - 5\cos(t) + \sin(t).$$

Using $N_r = 500$ points $t_i \sim \text{Unif}(0, 5)$, the residual loss is

$$\mathcal{L}_r = \frac{1}{N_r} \sum_{i=1}^{N_r} \left| \frac{du_\theta}{dt}(t_i) + 5u_\theta(t_i) - 5\cos(t_i) + \sin(t_i) \right|^2.$$

The initial condition loss is

$$\mathcal{L}_{ic} = |u_\theta(0)|^2.$$

Therefore, the full loss used for training is

$$\mathcal{L} = \mathcal{L}_r + 50\mathcal{L}_{ic}.$$

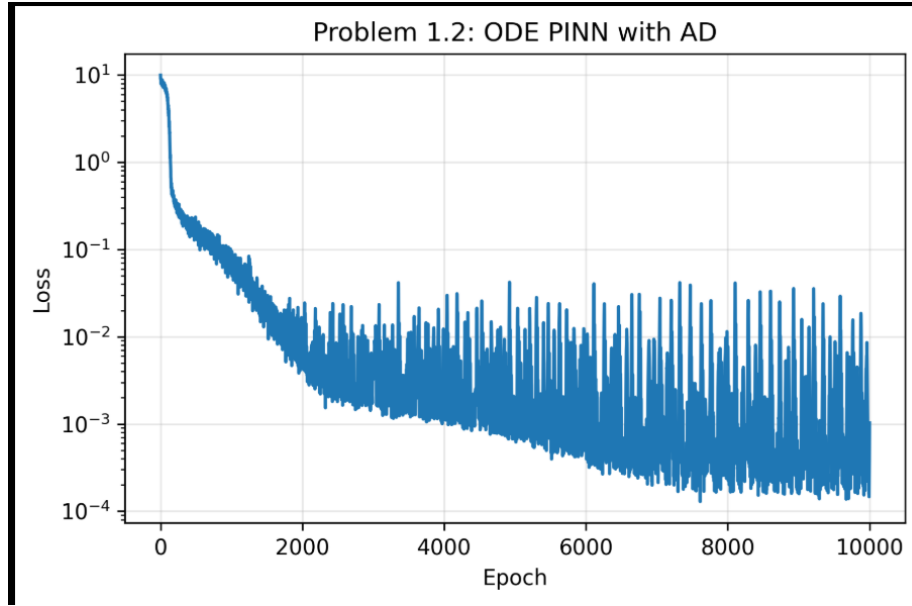
And we compute the derivative $\frac{du_q}{dt}$ is computed using automatic differentiation with `torch.autograd.grad`.

Part B

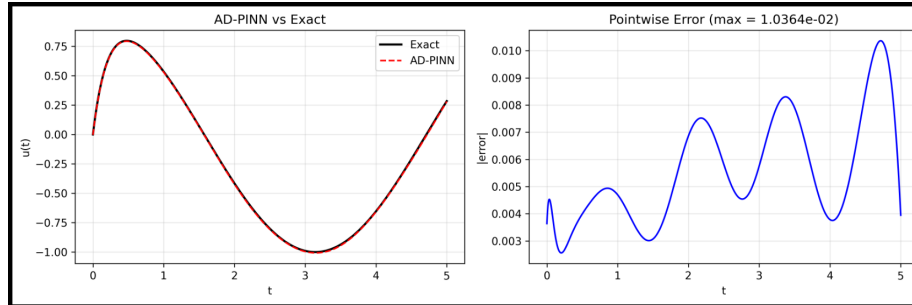
The AD-PINN was trained for 10,000 epochs. The final training loss and training time are below

```
$ py starter_code.py
=====
Problem 1.2: ODE PINN (Autograd)
=====
Epoch 2000/10000, Loss = 5.638965e-03
Epoch 4000/10000, Loss = 1.405145e-03
Epoch 6000/10000, Loss = 2.572970e-03
Epoch 8000/10000, Loss = 4.899329e-03
Epoch 10000/10000, Loss = 1.029281e-03
Training time: 30.5s
Max absolute error: 1.036367e-02

Problem 1.2 AD-PINN Summary
Final training loss: 1.029281e-03
Max absolute error on 1000 test points: 1.036367e-02
Wall-clock training time: 30.52 seconds
```



Training loss curve for the ODE AD-PINN, plotted on a logarithmic scale.



AD-PINN approximation compared with the exact solution $u(t) = \cos(t) - e^{-5t}$, together with the pointwise absolute error. The loss decreases rapidly during the early epochs and then continues to decrease more slowly, reaching a final value of 1.029281×10^{-3} . The spikes in the loss curve are expected because the points are sampled randomly during training. The comparison plot shows that the AD-PINN approximation closely matches the exact solution on $[0, 5]$. On 1000 test points, the maximum absolute error was 1.036367×10^{-2} , which indicates that the trained network learned the ODE solution reasonably well.

Problem 1.3

Part A

Following the structure provided, for the finite-difference PINN, the same ODE residual is used as in Problem 1.2, but we approximate the derivative using the central difference formula

$$\frac{du_\theta}{dt}(t_i) \approx \frac{u_\theta(t_i + \epsilon) - u_\theta(t_i - \epsilon)}{2\epsilon}.$$

Substituting this approximation into the residual gives

$$R_\theta(t_i) = \frac{u_\theta(t_i + \epsilon) - u_\theta(t_i - \epsilon)}{2\epsilon} + 5u_\theta(t_i) - 5\cos(t_i) + \sin(t_i).$$

Using $N_r = 500$ points $t_i \sim \text{Unif}(0, 5)$, the residual loss is

$$\mathcal{L}_r = \frac{1}{N_r} \sum_{i=1}^{N_r} \left| \frac{u_\theta(t_i + \epsilon) - u_\theta(t_i - \epsilon)}{2\epsilon} + 5u_\theta(t_i) - 5\cos(t_i) + \sin(t_i) \right|^2.$$

The initial condition loss is the same as before:

$$\mathcal{L}_{ic} = |u_\theta(0)|^2.$$

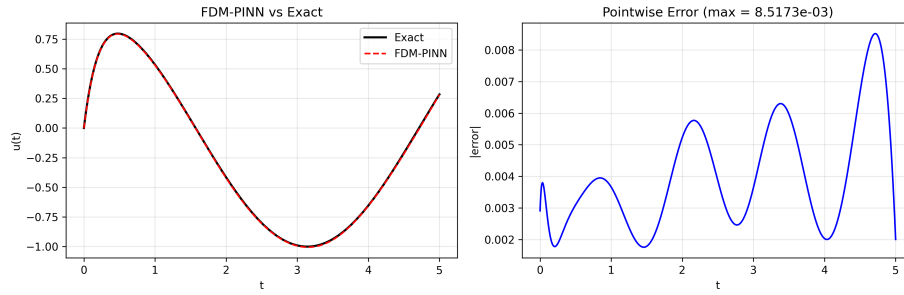
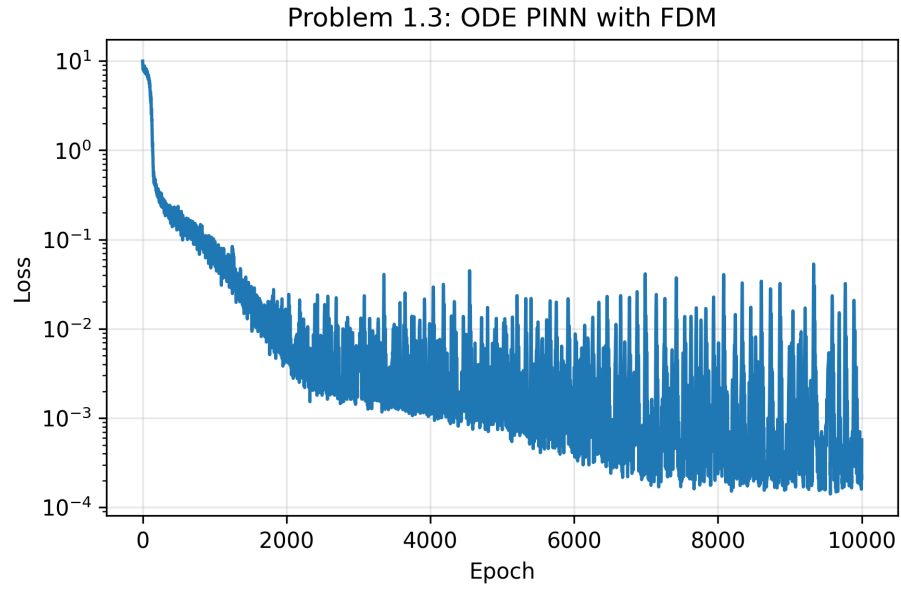
Therefore, the full finite-difference PINN loss is

$$\mathcal{L} = \mathcal{L}_r + 50\mathcal{L}_{ic}.$$

$\epsilon = 10^{-3}$ is used for the central difference approximation.

Part B

The FDM-PINN was trained for 10,000 epochs with $\epsilon = 10^{-3}$. The training loss curve on a logarithmic scale is shown below



The trained FDM-PINN was evaluated on 1000 evenly spaced test points in $[0, 5]$. We compares the neural network approximation with the exact solution

Problem 1.3: ODE PINN (FDM)

```

Epoch 2000/10000, Loss = 5.647130e-03
Epoch 4000/10000, Loss = 1.640812e-03
Epoch 6000/10000, Loss = 3.301742e-03
Epoch 8000/10000, Loss = 7.819664e-04
Epoch 10000/10000, Loss = 5.718605e-04
Training time: 36.3s
Max absolute error: 8.517336e-03

```

Problem 1.3 FDM-PINN Summary

```

Epsilon: 1.0e-03
Final training loss: 5.718605e-04
Max absolute error on 1000 test points: 8.517336e-03
Wall-clock training time: 36.28 seconds

```

The loss curve shows how the FDM-PINN training loss changes over the 10,000 training epochs. Since the points are sampled randomly, some oscillation in the loss curve is expected. The comparison plot shows whether the trained neural network successfully approximates the exact solution on $[0, 5]$.

Problem 1.4

Part A

The AD-PINN and FDM-PINN are compared using the final training loss, the maximum absolute error on 1000 evenly spaced test points in $[0, 5]$, and the training time.

Problem 1.4(a): AD-PINN vs FDM-PINN			
Method	Final Loss	Max Abs Error	Time (s)
AD-PINN	1.029281e-03	1.036367e-02	29.88
FDM-PINN	1.961547e-03	6.422609e-03	34.95

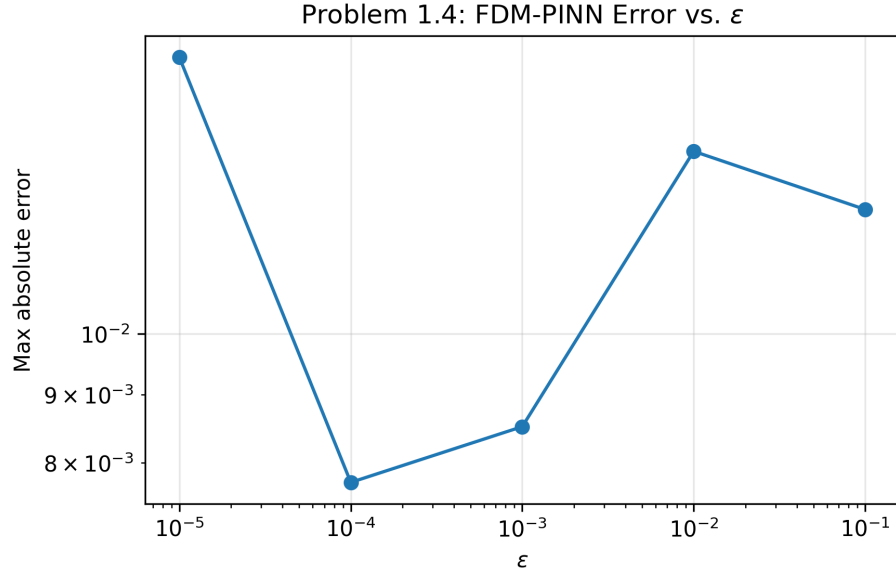
Compared here, this shows that both methods produce a small maximum error even if they compute $\frac{du_\theta}{dt}$ differently ,

Part B

The FDM-PINN was trained using

$$\epsilon \in \{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}.$$

For each value of ϵ , the network was trained for 10,000 epochs, and the maximum absolute error was computed on 1000 evenly spaced test points in $[0, 5]$. Below is maximum absolute error of the FDM-PINN as a function of ϵ .



Here is the effect of ϵ on the FDM-PINN error for the first-order ODE.

ϵ	Final Training Loss	Max Absolute Error	Training Time (s)
10^{-1}	1.928638×10^{-4}	1.239145×10^{-2}	3.404905×10^1
10^{-2}	2.670515×10^{-3}	1.370004×10^{-2}	3.525049×10^1
10^{-3}	5.718605×10^{-4}	8.517336×10^{-3}	3.571479×10^1
10^{-4}	6.258825×10^{-4}	7.736003×10^{-3}	3.517855×10^1
10^{-5}	5.635995×10^{-3}	1.611501×10^{-2}	3.431908×10^1

Among the tested values, $\epsilon = 10^{-4}$ gave the smallest maximum absolute error, 7.736003×10^{-3} . Therefore, in this experiment, $\epsilon = 10^{-4}$ was the best empirical choice. However, this does not prove that 10^{-4} is the exact optimal value of ϵ , since only five discrete values were tested and PINN training is affected by random collocation sampling and nonconvex optimization. For large ϵ , the finite-difference derivative approximation has larger truncation error because the central difference formula only approximates the derivative to order $O(\epsilon^2)$. Therefore, decreasing ϵ will initially improve the accuracy.

However, choosing ϵ too small does not always improve the result. When ϵ is very small, the two values $u_\theta(t_i + \epsilon)$ and $u_\theta(t_i - \epsilon)$ can become extremely close. Subtracting nearly equal floating-point numbers can produce round-off error and loss of significance. Dividing this small difference by 2ϵ can further amplify the round-off error. Therefore, the best ϵ is usually balanced between truncation error for large ϵ and floating-point round-off error for very small ϵ .

Problem 2.1

Part A

Let

$$x_j = j\Delta x, \quad j = 0, 1, \dots, 64,$$

where

$$\Delta x = \frac{1}{64}.$$

Also let

$$t_n = n\Delta t, \quad n = 0, 1, \dots, N_t.$$

I use U_j^n to represent the numerical approximation to the exact solution $u(x_j, t_n)$.

Using the standard Forward Euler finite-difference formula for the heat equation as shown in the textbook,

$$U_j^{n+1} = U_j^n + r (U_{j+1}^n - 2U_j^n + U_{j-1}^n),$$

where

$$r = \frac{\nu \Delta t}{(\Delta x)^2}.$$

The boundary values are fixed by the boundary conditions:

$$U_0^n = 0, \quad U_{64}^n = 0.$$

The initial values come from

$$U_j^0 = \sin(\pi x_j) + \frac{1}{2} \sin(3\pi x_j).$$

The explicit heat-equation scheme requires

$$r \leq \frac{1}{2}.$$

Since $\nu = 0.01$ and $\Delta x = 1/64$, this means

$$\frac{0.01\Delta t}{(1/64)^2} \leq \frac{1}{2}.$$

Solving for Δt ,

$$\Delta t \leq \frac{1}{2} \frac{(1/64)^2}{0.01}.$$

Thus,

$$\Delta t \leq \frac{1}{2} \frac{1/4096}{0.01} = 1.220703125 \times 10^{-2}.$$

To reach the final time $T = 0.5$ exactly, I choose N_t so that

$$\Delta t = \frac{T}{N_t} = \frac{0.5}{N_t}.$$

We need

$$\frac{0.5}{N_t} \leq 1.220703125 \times 10^{-2}.$$

Therefore,

$$N_t \geq \frac{0.5}{1.220703125 \times 10^{-2}} = 40.96.$$

Since N_t must be an integer, I take

$$N_t = \lceil 40.96 \rceil = 41.$$

Then

$$\Delta t = \frac{0.5}{41} = 1.219512 \times 10^{-2}.$$

With this choice,

$$r = \frac{0.01\Delta t}{(1/64)^2} = 4.995122 \times 10^{-1} < \frac{1}{2}.$$

Therefore, the time step satisfies the stability condition while still reaching $t = 0.5$ exactly.

Problem 2.1: Forward Euler Finite-Difference Heat Equation				
Delta x	Delta t	Nt	r	L2 Error
1.56250000e-02	1.21951220e-02	41	4.99512195e-01	3.65539694e-04
PS C:\Users\azhu3\Documents\Github\Numerical-Analysis-Final-Project>				

Part B

The exact solution is

$$u(x, t) = e^{-\nu\pi^2 t} \sin(\pi x) + \frac{1}{2} e^{-9\nu\pi^2 t} \sin(3\pi x).$$

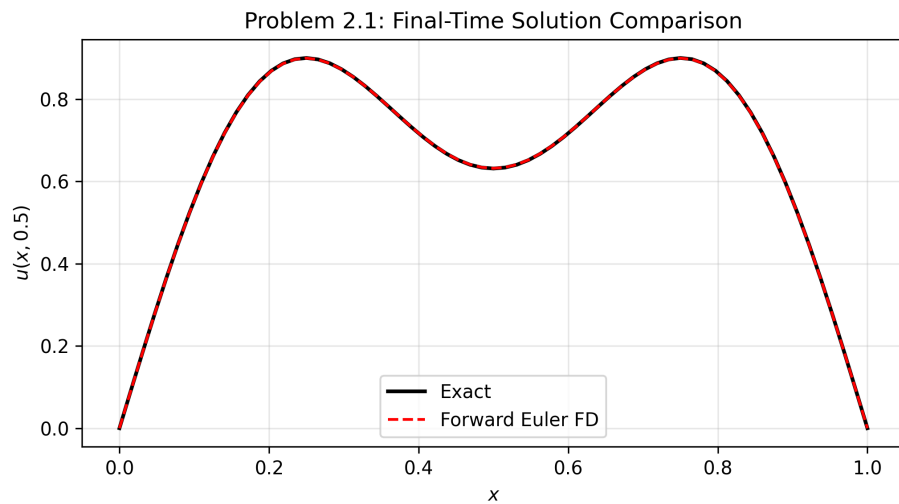
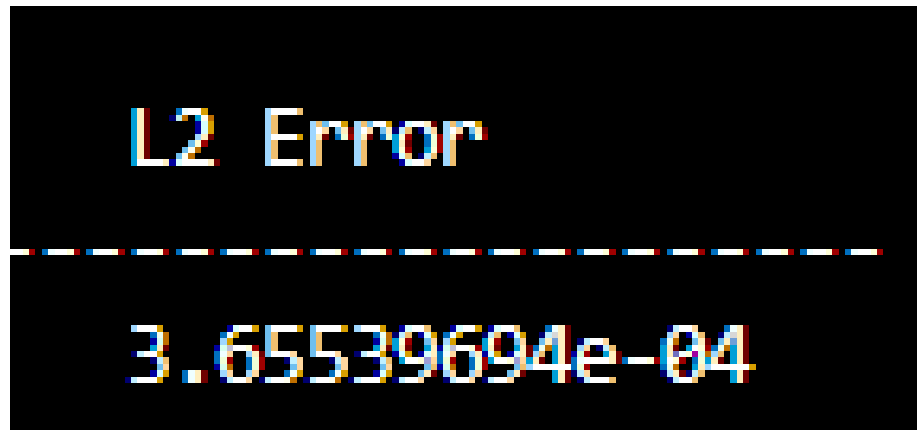
At $t = 0.5$, I computed the discrete L^2 error using

$$\|U - u\|_2 = \left(\Delta x \sum_{j=0}^{64} \left| U_j^{N_t} - u(x_j, 0.5) \right|^2 \right)^{1/2}.$$

This gave

$$\|U - u\|_2 \approx 3.655397 \times 10^{-4}.$$

As seen in my output

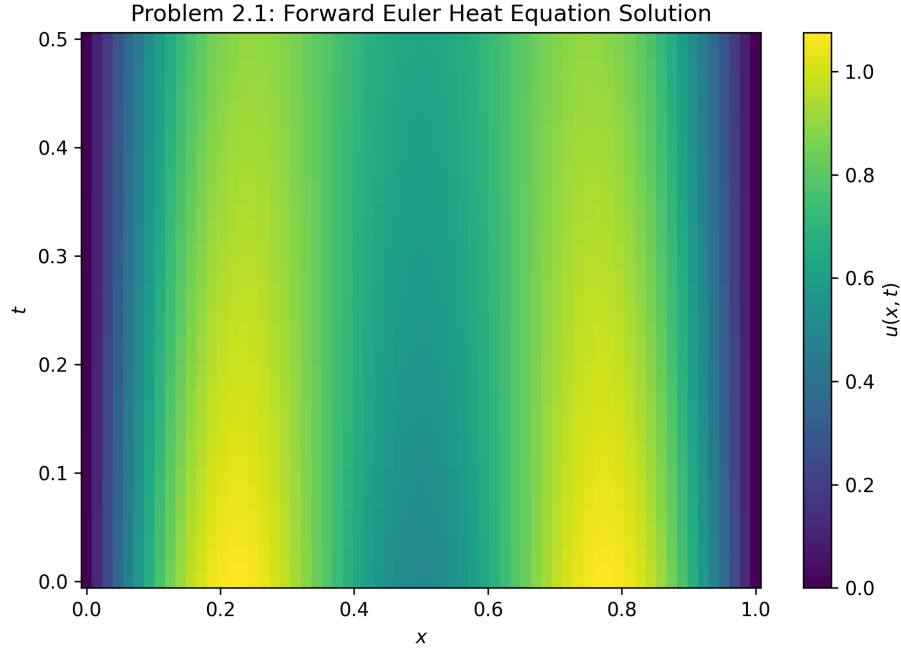


For consistency's sake in part 1 and to better help visualize this, I also have a plot that compares the Forward Euler finite-difference approximation with the exact solution at the final time $t = 0.5$. The black solid curve represents the exact solution while the red dashed curve represents the numerical solution computed by the finite-difference method.

The two curves almost overlap across the full interval $0 \leq x \leq 1$, which shows that the Forward Euler method gives an accurate approximation at the final time. The boundary conditions are also preserved, since both curves are equal to zero at $x = 0$ and $x = 1$. The incredibly small minor visible difference between the curves is consistent with the nonzero discrete L^2 error computed in Part B.

Part C

The numerical solution over the full space-time domain is shown below as the Forward Euler finite-difference solution of the heat equation over (x, t) . In the heatmap, the horizontal axis represents the spatial variable x , and the vertical axis represents time t . The color represents the value of the numerical solution $U_j^n \approx u(x_j, t_n)$, with lighter colors corresponding to larger values and darker colors corresponding to smaller values.



The plot reflects the boundary conditions

$$u(0, t) = u(1, t) = 0,$$

since the solution remains close to zero along the left and right edges of the domain. The initial condition contains two sine modes,

$$u(x, 0) = \sin(\pi x) + \frac{1}{2} \sin(3\pi x),$$

which explains the two main high-value regions visible near $x \approx 0.25$ and $x \approx 0.75$. As time increases, the heat equation smooths the solution. This smoothing effect is seen in the heatmap because the colors become slightly less intense as t approaches 0.5. Since $\nu = 0.01$ is relatively small, the diffusion is gradual rather than extremely rapid.

Overall, the heatmap shows that the Forward Euler method preserves the zero boundary conditions and evolves the initial temperature profile smoothly over time, as expected for the heat equation.

Problem 2.2

Part A

Following the structure provided, for the heat equation PINN, the neural network takes two inputs, x and t , and outputs an approximation

$$u_\theta(x, t) \approx u(x, t).$$

The PDE is

$$u_t = 0.01u_{xx},$$

so the residual used in the physics loss is

$$r_\theta(x, t) = \frac{\partial u_\theta}{\partial t}(x, t) - 0.01 \frac{\partial^2 u_\theta}{\partial x^2}(x, t).$$

I sampled $N_r = 10000$ interior collocation points from

$$(x, t) \in (0, 1) \times (0, 0.5].$$

Both x and t were set to require gradients, so that PyTorch could compute the needed derivatives.

First, the network output was evaluated as

$$u_\theta(x_i, t_i).$$

Then I used automatic differentiation to compute

$$\frac{\partial u_\theta}{\partial x} \quad \text{and} \quad \frac{\partial u_\theta}{\partial t}.$$

A second automatic differentiation call was then applied to $\partial u_\theta / \partial x$ in order to compute

$$\frac{\partial^2 u_\theta}{\partial x^2}.$$

Therefore, the residual loss was

$$L_r = \frac{1}{N_r} \sum_{i=1}^{N_r} \left| \frac{\partial u_\theta}{\partial t}(x_i, t_i) - 0.01 \frac{\partial^2 u_\theta}{\partial x^2}(x_i, t_i) \right|^2.$$

The initial condition is

$$u(x, 0) = \sin(\pi x) + \frac{1}{2} \sin(3\pi x).$$

Using $N_{ic} = 200$ initial-condition points, the initial-condition loss was then

$$L_{ic} = \frac{1}{N_{ic}} \sum_{i=1}^{N_{ic}} \left| u_\theta(x_i, 0) - \left(\sin(\pi x_i) + \frac{1}{2} \sin(3\pi x_i) \right) \right|^2.$$

The boundary conditions are

$$u(0, t) = 0, \quad u(1, t) = 0.$$

Using $N_{bc} = 200$ boundary points for each boundary, the boundary-condition loss was

$$L_{bc} = \frac{1}{N_{bc}} \sum_{i=1}^{N_{bc}} |u_{\theta}(0, t_i)|^2 + \frac{1}{N_{bc}} \sum_{i=1}^{N_{bc}} |u_{\theta}(1, t_i)|^2.$$

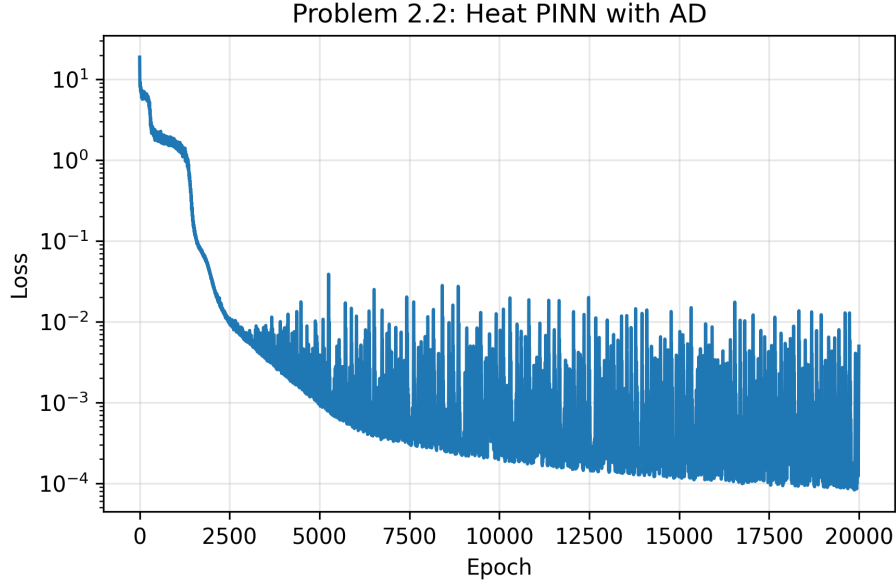
The final loss function was therefore

$$L = L_r + 20L_{ic} + 20L_{bc}.$$

The weights on L_{ic} and L_{bc} force the network to satisfy the initial and boundary conditions more strongly while still minimizing the PDE residual in the interior.

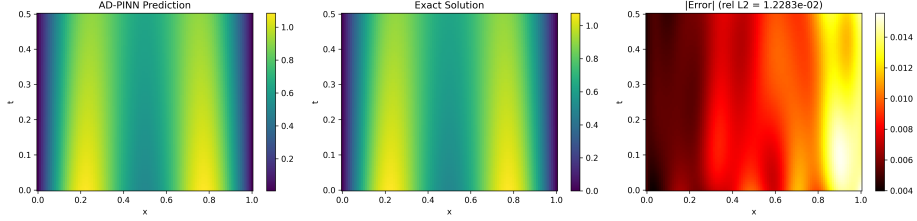
Part B

The AD-PINN was trained for 20000 epochs and compared against the exact solution



The training loss curve for the heat equation AD-PINN decreases substantially during training, showing that the network is learning to satisfy the heat equation residual, the initial condition, and the boundary conditions at the sampled collocation points. Since the loss is plotted on a logarithmic scale, the downward trend indicates that the error is decreasing over several orders of magnitude during training.

The loss decreases rapidly during the first several thousand epochs and then levels off at a much smaller value. The curve is not perfectly smooth because new random collocation points are sampled during training, so the computed loss can fluctuate from epoch to epoch. The spikes in the later part of training are therefore expected, but the overall downward trend shows that the AD-PINN is successfully reducing the physics-informed loss.



Above is the AD-PINN prediction, exact solution, and pointwise error heatmap for the heat equation. The predicted solution heatmap has the same general structure as the exact solution. The solution starts from the initial profile

$$\sin(\pi x) + \frac{1}{2} \sin(3\pi x)$$

at $t = 0$, and then smooths out over time due to diffusion. The boundary values remain close to zero at $x = 0$ and $x = 1$, which shows that the boundary-condition loss is being enforced successfully.

The pointwise error heatmap shows where the PINN prediction differs most from the exact solution. The largest errors are occurring near regions where the solution changes more rapidly, especially near the initial condition. This is expected because the network must simultaneously fit the initial profile, satisfy the zero boundary conditions, and satisfy the PDE residual throughout the interior of the domain.

```
=====
Problem 2.2: Heat PINN (Autograd)
=====

Epoch 2000/20000, Loss = 3.443553e-02
Epoch 4000/20000, Loss = 2.350010e-03
Epoch 6000/20000, Loss = 1.078987e-03
Epoch 8000/20000, Loss = 5.222651e-04
Epoch 10000/20000, Loss = 2.666970e-04
Epoch 12000/20000, Loss = 2.429449e-03
Epoch 14000/20000, Loss = 8.362625e-04
Epoch 16000/20000, Loss = 1.206901e-03
Epoch 18000/20000, Loss = 1.140707e-04
Epoch 20000/20000, Loss = 4.976995e-03
Training time: 577.7s
Relative L2 error: 1.228341e-02

Problem 2.2 AD-PINN Heat Summary
Final training loss: 4.976995e-03
Relative L2 error on 100 x 100 test grid: 1.228341e-02
Wall-clock training time: 577.65 seconds
```

On the 100×100 test grid, the relative L^2 error was

$$\frac{\|u_\theta - u\|_2}{\|u\|_2} = 1.228341 \times 10^{-2}.$$

The final training loss was

$$\text{Final loss} = 4.976995 \times 10^{-3}.$$

Overall, the AD-PINN successfully approximates the heat equation solution as the network learns a continuous approximation $u_\theta(x, t)$, and automatic differentiation gives the needed derivatives u_t and u_{xx} directly from the neural network.

Problem 2.3

Part A

As per the guideline suggestion, for the finite-difference PINN, the neural network again takes two inputs, x and t , and outputs an approximation

$$u_\theta(x, t) \approx u(x, t).$$

The heat equation is

$$u_t = 0.01u_{xx},$$

so the PDE residual is

$$r_\theta(x, t) = \frac{\partial u_\theta}{\partial t}(x, t) - 0.01 \frac{\partial^2 u_\theta}{\partial x^2}(x, t).$$

In this part, instead of using automatic differentiation to compute the derivatives, we approximated them using central finite differences. For the time derivative, we use

$$\frac{\partial u_\theta}{\partial t}(x_i, t_i) \approx \frac{u_\theta(x_i, t_i + \epsilon) - u_\theta(x_i, t_i - \epsilon)}{2\epsilon}.$$

For the second spatial derivative, we used

$$\frac{\partial^2 u_\theta}{\partial x^2}(x_i, t_i) \approx \frac{u_\theta(x_i + \epsilon, t_i) - 2u_\theta(x_i, t_i) + u_\theta(x_i - \epsilon, t_i)}{\epsilon^2}.$$

For the main experiment, we have $\epsilon = 10^{-3}$.

The PDE residual loss was therefore

$$L_r = \frac{1}{N_r} \sum_{i=1}^{N_r} \left| \frac{u_\theta(x_i, t_i + \epsilon) - u_\theta(x_i, t_i - \epsilon)}{2\epsilon} - 0.01 \frac{u_\theta(x_i + \epsilon, t_i) - 2u_\theta(x_i, t_i) + u_\theta(x_i - \epsilon, t_i)}{\epsilon^2} \right|^2.$$

I used $N_r = 10000$ interior collocation points. To keep the finite-difference stencil inside the domain, the residual points were sampled away from the boundaries by a margin of ϵ .

The initial-condition loss was the same as in the AD-PINN case. Since

$$u(x, 0) = \sin(\pi x) + \frac{1}{2} \sin(3\pi x),$$

I used

$$L_{ic} = \frac{1}{N_{ic}} \sum_{i=1}^{N_{ic}} \left| u_\theta(x_i, 0) - \left(\sin(\pi x_i) + \frac{1}{2} \sin(3\pi x_i) \right) \right|^2,$$

with $N_{ic} = 200$.

The boundary conditions are

$$u(0, t) = 0, \quad u(1, t) = 0.$$

Thus the boundary-condition loss was

$$L_{bc} = \frac{1}{N_{bc}} \sum_{i=1}^{N_{bc}} |u_\theta(0, t_i)|^2 + \frac{1}{N_{bc}} \sum_{i=1}^{N_{bc}} |u_\theta(1, t_i)|^2,$$

with $N_{bc} = 200$ points on each boundary.

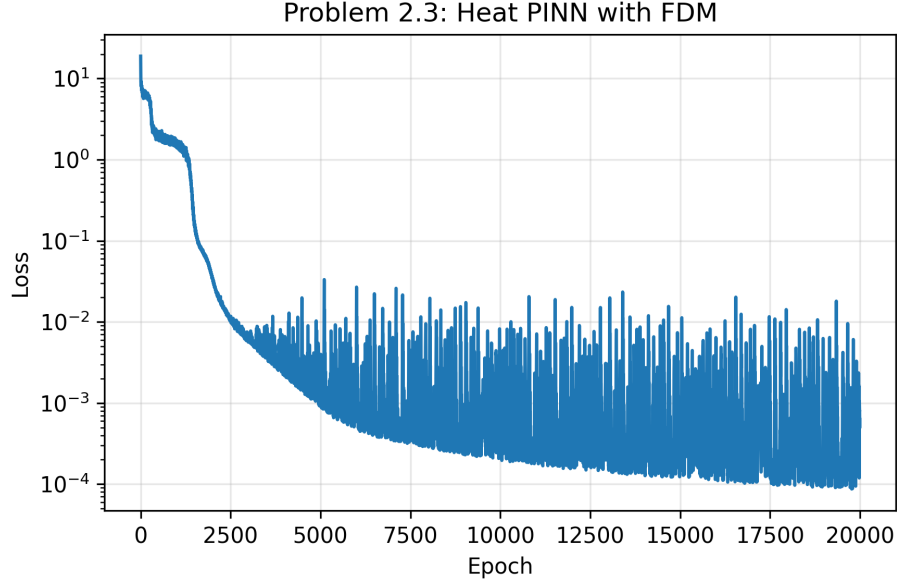
The total loss was

$$L = L_r + 20L_{ic} + 20L_{bc}.$$

This loss has the same structure as the AD-PINN loss, but the PDE derivatives are approximated using finite differences instead of automatic differentiation.

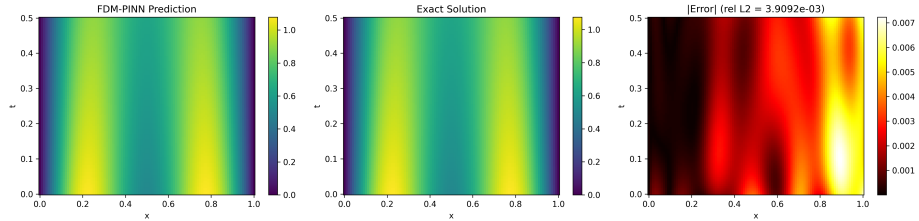
Part B

The FDM-PINN was trained for 20000 epochs with $\epsilon = 10^{-3}$. and compared against exact solution



The training loss curve for the heat equation FDM-PINN shows that the physics-informed loss decreases substantially during training. Since the loss includes the PDE residual, initial-condition error, and boundary-condition error, the decreasing loss indicates that the neural network is improving its approximation of the heat equation solution while also satisfying the initial and boundary conditions.

The loss decreases rapidly during the first several thousand epochs and then levels off around a much smaller value. The curve is not perfectly smooth because new random collocation points are sampled during training, so the computed loss can fluctuate from epoch to epoch. The spikes in the later part of training are therefore expected, but the overall downward trend on the logarithmic scale shows that the method is successfully reducing the error over time.



Above is the FDM-PINN prediction, exact solution, and pointwise error heatmap for the heat equation. The predicted solution heatmap has nearly the same structure as the exact solution. The solution begins from the initial profile

$$\sin(\pi x) + \frac{1}{2} \sin(3\pi x)$$

at $t = 0$, and then gradually smooths over time because of diffusion. Since $\nu = 0.01$ is relatively small and the final time is only $t = 0.5$, the overall shape of the solution remains clearly visible. The values near $x = 0$ and $x = 1$ remain close to zero, showing that the boundary conditions are being enforced.

The pointwise error heatmap shows that the FDM-PINN prediction is close to the exact solution over the full space-time domain. The largest errors are on the order of 10^{-3} , with the maximum visible error around 7×10^{-3} . In this run, the largest errors appear mostly toward the right side of the domain, especially near larger values of x . This is still small compared with the solution magnitude, which is about order 1. In this method, the derivative approximation also depends on the finite-difference parameter ϵ . If ϵ is too large, the derivative approximation has more truncation error. If ϵ is too small, round-off error can become more important, especially because the second derivative formula divides by ϵ^2 . Compared with the AD-PINN in Problem 2.2, the FDM-PINN gives a similar overall solution shape, but its derivative information comes from finite-difference approximations rather than exact automatic differentiation of the network. As a result, the FDM-PINN error is also affected by the choice of ϵ , so its accuracy depends not only on the network training but also on the truncation-error and round-off-error trade-off in the finite-difference derivative estimates.

```

=====
Problem 2.3: Heat PINN (FDM)
=====
Epoch 2000/20000, Loss = 3.470736e-02
Epoch 4000/20000, Loss = 2.701890e-03
Epoch 6000/20000, Loss = 1.062871e-02
Epoch 8000/20000, Loss = 2.953266e-04
Epoch 10000/20000, Loss = 8.292751e-04
Epoch 12000/20000, Loss = 4.970761e-03
Epoch 14000/20000, Loss = 2.333480e-04
Epoch 16000/20000, Loss = 3.525646e-03
Epoch 18000/20000, Loss = 1.473490e-04
Epoch 20000/20000, Loss = 6.265717e-04
Training time: 520.2s
Relative L2 error: 3.909162e-03

Problem 2.3 FDM-PINN Heat Summary
Epsilon: 1.0e-03
Final training loss: 6.265717e-04
Relative L2 error on 100 x 100 test grid: 3.909162e-03
Wall-clock training time: 520.18 seconds

```

On the 100×100 test grid, the relative L^2 error was

$$\frac{\|u_\theta - u\|_2}{\|u\|_2} = 3.909162 \times 10^{-3}.$$

The final training loss was

$$\text{Final loss} = 6.265717 \times 10^{-4}.$$

The wall-clock training time was

$$520.18 \text{ seconds.}$$

Overall, the FDM-PINN successfully approximates the heat equation solution using finite-difference derivative estimates inside the physics loss. Compared with the AD-PINN, this method avoids computing u_t and u_{xx} through automatic differentiation, but it introduces a new numerical parameter ϵ . Therefore, the accuracy depends not only on the network and training process, but also on the finite-difference balance between truncation error and round-off error.

Problem 2.4

Part A

The AD-PINN and FDM-PINN were compared using final training loss, relative L^2 error on the 100×100 test grid, and wall-clock training time. The comparison is shown in the table below.

Problem 2.4: Comparison and Stability			
Problem 2.4(a): Heat AD-PINN vs Heat FDM-PINN			
Method	Final Loss	Relative L2 Error	Time (s)
AD-PINN	3.679594e-04	3.546381e-03	909.01
FDM-PINN	1.614389e-03	7.299632e-03	516.06

Both methods successfully approximate the heat equation solution. The AD-PINN computes the derivatives u_t and u_{xx} directly through automatic differentiation, while the FDM-PINN approximates these derivatives using finite differences. In this experiment, the AD-PINN produced both a smaller final training loss and a smaller relative L^2 error than the FDM-PINN. The AD-PINN had a relative L^2 error of

$$3.546381 \times 10^{-3},$$

while the FDM-PINN had a relative L^2 error of

$$7.299632 \times 10^{-3}.$$

Thus, for this run, the AD-PINN was more accurate.

However, the AD-PINN also required more wall-clock training time. It took 909.01 seconds, compared with 516.06 seconds for the FDM-PINN. This is reasonable because the AD-PINN computes derivatives through the neural network computational graph, including the second derivative u_{xx} , which is relatively expensive. The FDM-PINN avoids automatic differentiation for the PDE derivatives by using finite-difference approximations, making it faster in this experiment.

The AD-PINN has the advantage that it does not require choosing a finite-difference step size for the PDE derivatives. This avoids the truncation-error and round-off-error tradeoff associated with finite differences. The FDM-PINN is conceptually closer to classical numerical differentiation, but its accuracy depends on the choice of ϵ . If ϵ is too large, truncation error can increase; if ϵ is too small, round-off error can become more important, especially because the second derivative formula divides by ϵ^2 . Overall, the AD-PINN was more accurate in this run, while the FDM-PINN was faster.

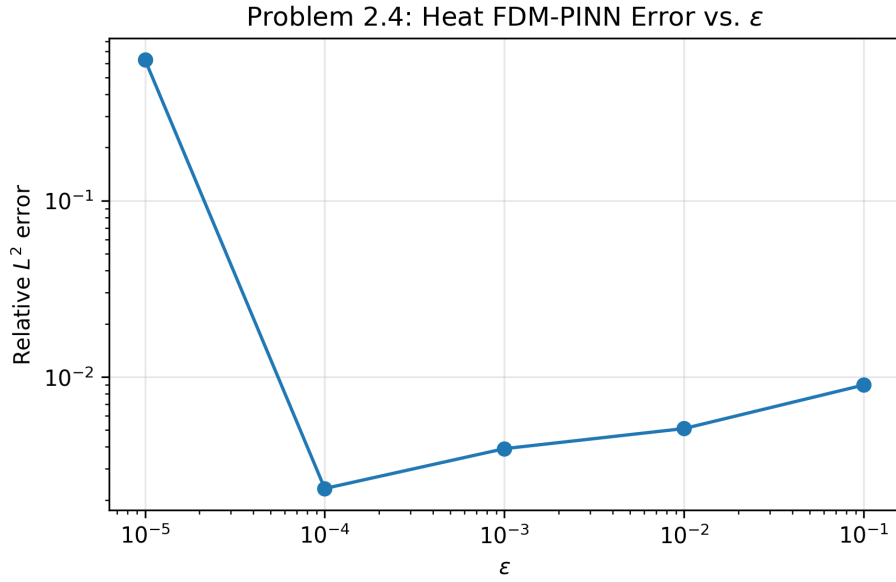
Part B

For the FDM-PINN, I varied

$$\epsilon \in \{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}$$

and trained a separate FDM-PINN for each value of ϵ . The relative L^2 error was then computed on the same 100×100 test grid. The numerical results are shown below. and we can see the effect of ϵ on the heat equation FDM-PINN.

ϵ	Final Loss	Relative L^2 Error	Time (s)
10^{-1}	9.014032×10^{-4}	8.982802×10^{-3}	478.24
10^{-2}	8.722325×10^{-4}	5.084104×10^{-3}	463.28
10^{-3}	6.265717×10^{-4}	3.909162×10^{-3}	477.84
10^{-4}	1.553886×10^{-2}	2.323322×10^{-3}	491.44
10^{-5}	8.221640	6.284757×10^{-1}	510.54



Above is the relative L^2 error versus ϵ for the heat equation FDM-PINN. The plot shows a clear dependence of the FDM-PINN accuracy on the finite-difference parameter ϵ . As ϵ decreases from 10^{-1} to 10^{-4} , the relative L^2 error decreases from

$$8.982802 \times 10^{-3}$$

to

$$2.323322 \times 10^{-3}.$$

This suggests that, over this range, reducing ϵ improves the finite-difference derivative approximation by lowering truncation error.

However, when ϵ is decreased further to 10^{-5} , the relative L^2 error increases sharply to

$$6.284757 \times 10^{-1}.$$

This shows that making ϵ too small does not necessarily improve the method. For very small ϵ , the finite-difference formulas subtract nearly equal neural network outputs and then divide by very small numbers. This makes round-off error and numerical cancellation much more significant.

This tradeoff is especially important for the heat equation because the PDE residual requires the second derivative approximation

$$\frac{\partial^2 u_\theta}{\partial x^2}(x_i, t_i) \approx \frac{u_\theta(x_i + \epsilon, t_i) - 2u_\theta(x_i, t_i) + u_\theta(x_i - \epsilon, t_i)}{\epsilon^2}.$$

The division by ϵ^2 makes the approximation more sensitive to small ϵ than a first-derivative formula, which only divides by ϵ . Therefore, the truncation-error versus round-off-error tradeoff is more pronounced for the heat equation than for the ODE problem.

In this experiment, the best value of ϵ , measured by relative L^2 error, was

$$\epsilon = 10^{-4}.$$

At this value, the relative L^2 error was

$$2.323322 \times 10^{-3}.$$

The training times were all similar, ranging from about 463 seconds to 511 seconds, so the main effect of ϵ was on accuracy rather than computational cost.

Part C

For this part, I reran the Forward Euler finite-difference method for the heat equation using a time step that violates the CFL stability condition. For the explicit Forward Euler scheme,

$$r = \frac{\nu \Delta t}{(\Delta x)^2}$$

must satisfy

$$r \leq \frac{1}{2}$$

for stability. In this experiment, I set

$$r = 0.6,$$

which violates this condition.

Since

$$\Delta x = \frac{1}{64} = 1.562500 \times 10^{-2},$$

and $\nu = 0.01$, the time step was chosen from

$$\Delta t = \frac{r(\Delta x)^2}{\nu}.$$

Thus,

$$\Delta t = \frac{0.6(1.562500 \times 10^{-2})^2}{0.01} = 1.464844 \times 10^{-2}.$$

This gave

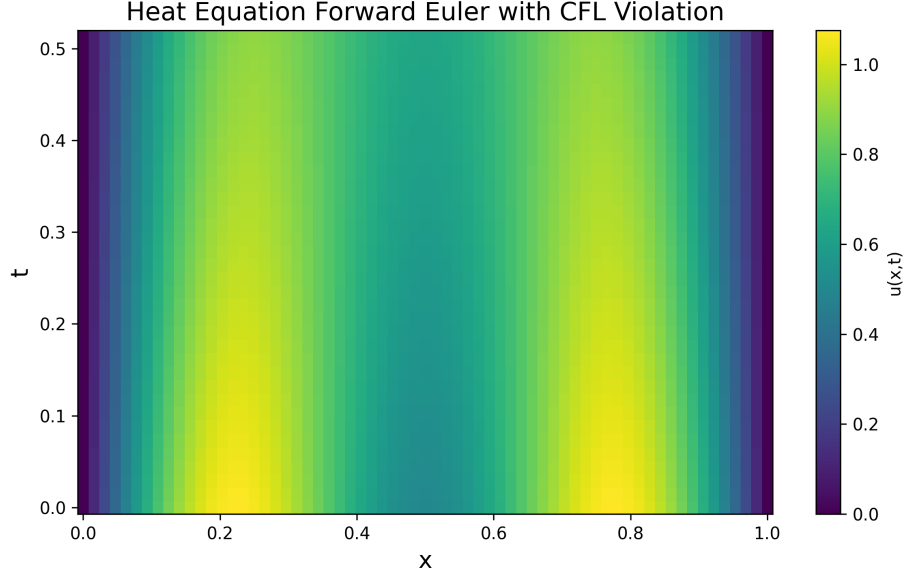
$$N_t = 35$$

time steps and a final computed time of approximately

$$t = 5.126953 \times 10^{-1}.$$

Problem 2.4(c): Forward Euler Finite-Difference Heat Equation with CFL Violation					
Delta x	Delta t	Nt	r	Final time	Max U
1.56250000e-02	1.46484375e-02	35	6.00000000e-01	5.12695312e-01	1.07535392e+00

Above is the Forward Euler finite-difference heat equation parameters with CFL violation.



This is the forward Euler finite-difference heat equation solution with $r = 0.6$, violating the CFL condition. The heatmap does not immediately show a dramatic blow-up. The solution still appears visually smooth over this short time interval, and the maximum value is about

$$\max |U| = 1.075354.$$

This happens because the initial condition is smooth and mainly contains low-frequency sine modes. The unstable behavior of the Forward Euler heat scheme usually appears most strongly in high-frequency modes, so the instability may not be visually dramatic over only 35 time steps.

However, the method is still unstable in general when $r > 1/2$. For the explicit Forward Euler finite-difference scheme, high-frequency error modes can grow instead of decay when the CFL condition is violated. Therefore, even though this particular plot does not immediately explode, the choice $r = 0.6$ is not stable in the theoretical sense.

The PINN methods do not suffer from the same CFL restriction because they are not explicit time-stepping methods. Instead of updating a grid solution from one time level to the next, the PINNs train a neural network $u_\theta(x, t)$ to satisfy the PDE residual, the initial condition, and the boundary conditions over sampled space-time points. Therefore, there is no explicit stability ratio

$$r = \frac{\nu \Delta t}{(\Delta x)^2}$$

that must be kept below $1/2$.

However, this does not mean PINNs have no numerical difficulties. PINNs can still be sensitive to optimization choices, collocation sampling, network

size, learning rate, and training time. For the FDM-PINN, the finite-difference parameter ϵ also introduces a truncation-error versus round-off-error tradeoff. Thus, PINNs avoid the classical CFL time-stepping restriction, but they replace it with different training and approximation challenges.

Problem 3

Part A

The errors of the classical numerical methods and the PINN methods are summarized in the table below. For the ODE problem, all methods are compared using maximum absolute error over the test grid. For the heat equation, the Forward Euler finite-difference method is measured using the discrete L^2 error at $t = 0.5$, while the PINN methods are measured using the relative L^2 error on a 100×100 test grid. Therefore, the ODE rows are directly comparable with each other, and the heat PINN rows are directly comparable with each other, but the heat FD error uses a slightly different L^2 -type metric.

Section 3(a): Error Comparison Across All Methods				
Problem	Method	Error Metric	Error	Time (s)
ODE	Forward Euler	Max absolute error	1.002978e-02	0.00
ODE	RK4	Max absolute error	2.164679e-08	0.00
ODE	AD-PINN	Max absolute error	1.036367e-02	28.77
ODE	FDM-PINN	Max absolute error	6.422609e-03	34.67
Heat	Forward Euler FD	L2 error at t=0.5	3.655397e-04	0.00
Heat	AD-PINN	Relative L2 error	3.546381e-03	909.01
Heat	FDM-PINN	Relative L2 error	7.299632e-03	516.06

The classical methods are the easiest to implement for these simple problems and are also the fastest. For the ODE, Forward Euler is simple and computationally cheap, but it is only first-order accurate, so its error is much larger than RK4 for the same step size. RK4 gives a much smaller error, 2.164679×10^{-8} , because it uses four slope evaluations per step and has fourth-order accuracy. In this experiment, RK4 gives the best balance of accuracy and computational cost for the ODE.

The ODE PINN methods are more computationally expensive than the classical methods. The AD-PINN gives a maximum absolute error of 1.036367×10^{-2} , while the FDM-PINN gives a smaller error of 6.422609×10^{-3} . However, both take much longer than Forward Euler or RK4 because they require neural network training over many epochs. The FDM-PINN also introduces the finite-difference parameter ϵ , while the AD-PINN avoids this extra derivative step-size choice.

For the heat equation, the Forward Euler finite-difference method is very fast and gives a small final-time L^2 error of 3.655397×10^{-4} . However, this method requires a fixed spatial and temporal mesh and must satisfy the CFL

stability condition

$$r = \frac{\nu \Delta t}{(\Delta x)^2} \leq \frac{1}{2}.$$

This makes the method easy to analyze, but it also restricts the allowable time step. If the time step is too large, the finite-difference solution can become unstable.

The heat equation PINNs are more flexible because they learn a continuous approximation $u_\theta(x, t)$ instead of solving only at mesh points. The AD-PINN achieved a relative L^2 error of 3.546381×10^{-3} , while the FDM-PINN achieved a relative L^2 error of 7.299632×10^{-3} . In this run, the AD-PINN was more accurate, but it also took longer to train. This makes sense because AD computes the needed derivatives through the computational graph, including the second derivative u_{xx} , which is computationally expensive.

Overall, the classical methods are faster and easier for low-dimensional problems with simple domains and known grids. The PINN methods are more general and mesh-free, but they require much more computation and their accuracy depends on training quality, network size, collocation sampling, and optimization. Among the classical ODE methods, RK4 was the most accurate. For the heat equation, the finite-difference method was fastest, while the PINN methods offered a continuous learned solution but at a much higher computational cost.

Part B

Note on hardware: Starting from Problem 3(b), I ran the remaining PINN experiments on a computer with GPU acceleration. Earlier timing results in the report were produced on CPU, so the wall-clock times from the earlier sections should not be directly compared with the GPU timings in Problems 3(b), 3(c), and the Bonus. However, within Problems 3(b) and 3(c), the runs were performed using the same GPU environment, so the relative timing comparisons inside those sections are still meaningful. The reported error values are still comparable across experiments, but the wall-clock training times should be interpreted with the hardware environment in mind.

For this part, I tested how the number of residual collocation points N_r affects the final accuracy of the AD-PINN and FDM-PINN methods. For the ODE problem, I used

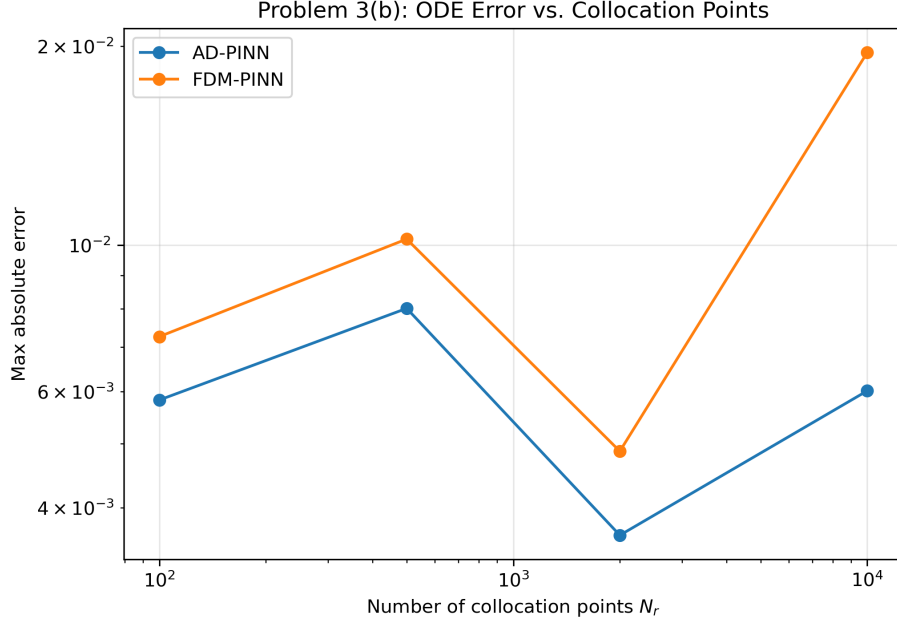
$$N_r \in \{100, 500, 2000, 10000\},$$

and for the heat equation, I used

$$N_r \in \{500, 2000, 10000, 50000\}.$$

as the assignment suggested. For each value of N_r , I trained a new model and evaluated the final error on a fixed test set. For the ODE, the error metric is the maximum absolute error on 1000 test points. For the heat equation, the error

metric is the relative L^2 error on a 100×100 test grid. Below is the ODE final error versus the number of collocation points N_r for AD-PINN and FDM-PINN.

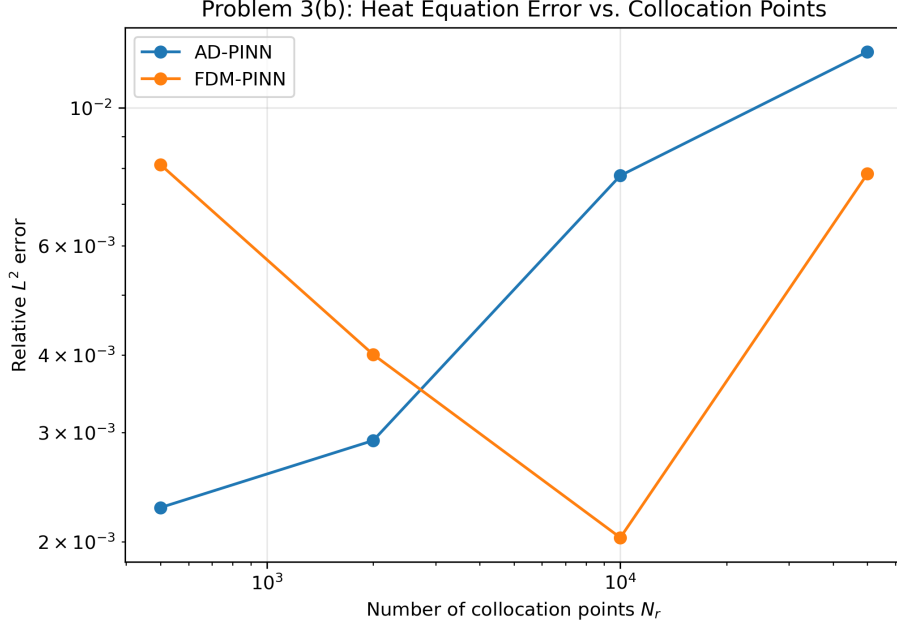


And a table of the ODE collocation-point sweep results which can be found in txt as well as terminal

Method	N_r	Final Loss	Max Error	Time (s)
AD-PINN	100	3.6631×10^{-4}	5.8225×10^{-3}	29.92
AD-PINN	500	3.9203×10^{-4}	8.0168×10^{-3}	29.76
AD-PINN	2000	2.4633×10^{-4}	3.6329×10^{-3}	29.64
AD-PINN	10000	2.2743×10^{-4}	6.0135×10^{-3}	29.28
FDM-PINN	100	1.2945×10^{-3}	7.2635×10^{-3}	33.75
FDM-PINN	500	1.9806×10^{-3}	1.0210×10^{-2}	32.44
FDM-PINN	2000	2.1497×10^{-4}	4.8701×10^{-3}	34.11
FDM-PINN	10000	1.0120×10^{-2}	1.9583×10^{-2}	30.15

For the ODE problem, increasing N_r did not lead to a perfectly monotone decrease in error. The AD-PINN obtained its smallest error at $N_r = 2000$, while the FDM-PINN also improved at $N_r = 2000$ but became worse at $N_r = 10000$. This shows that using more collocation points gives a better sampling of the residual loss, but it does not automatically guarantee a smaller final test error after a fixed number of training epochs. The final error is also affected by optimization behavior, random collocation sampling, and the finite-difference derivative approximation in the FDM-PINN.

Below is the heat equation final error versus the number of collocation points N_r for AD-PINN and FDM-PINN.



Here is the table of the heat-equation collocation-point sweep results which can be found in txt as well as terminal

Method	N_r	Final Loss	Relative L^2 Error	Time (s)
AD-PINN	500	2.4494×10^{-4}	2.2716×10^{-3}	109.62
AD-PINN	2000	3.5273×10^{-4}	2.9132×10^{-3}	131.28
AD-PINN	10000	6.7745×10^{-3}	7.7794×10^{-3}	169.42
AD-PINN	50000	1.2573×10^{-2}	1.2312×10^{-2}	171.94
FDM-PINN	500	2.4777×10^{-3}	8.1034×10^{-3}	158.70
FDM-PINN	2000	5.4615×10^{-4}	4.0062×10^{-3}	125.41
FDM-PINN	10000	4.7178×10^{-4}	2.0319×10^{-3}	128.52
FDM-PINN	50000	2.2440×10^{-3}	7.8316×10^{-3}	156.06

For the heat equation, the same nonmonotone behavior appears. The AD-PINN had the lowest relative L^2 error at $N_r = 500$, while the FDM-PINN performed best at $N_r = 10000$. Increasing N_r generally gives the residual loss a more accurate sampling of the space-time domain, but very large N_r also increases the computational cost per epoch and does not necessarily improve the trained solution under a fixed number of epochs.

This relates to quadrature accuracy because the residual loss is an empirical approximation to an integral of the squared residual over the domain. For

example, if

$$R_\theta(t) = \frac{du_\theta}{dt} + 5u_\theta(t) - 5\cos(t) + \sin(t),$$

then the continuous residual objective is analogous to

$$\int_D |R_\theta(t)|^2 dt,$$

while the PINN actually minimizes the sampled approximation

$$\frac{1}{N_r} \sum_{i=1}^{N_r} |R_\theta(t_r^{(i)})|^2.$$

Thus, changing N_r changes the accuracy of the numerical approximation to this residual integral.

This is similar in spirit to the numerical integration methods from class. In Newton–Cotes formulas, we approximate an integral by evaluating the function at a finite set of nodes, often equally spaced, and then taking a weighted sum. Gaussian quadrature improves on this idea by choosing both the nodes and weights more carefully; with m points, it can achieve degree of precision $2m - 1$, which is higher than the comparable Newton–Cotes rule. In my PINN experiments, the collocation points are random rather than optimized Gaussian nodes, so the residual loss is closer to a Monte Carlo-esque quadrature rule than to Gaussian quadrature. Increasing N_r should reduce sampling error in the residual-loss approximation, but it does not guarantee lower final solution error because the network parameters are also changing during training.

Therefore, the nonmonotone behavior in the plots is reasonable. More collocation points make the residual loss a better approximation of the continuous physics loss, but the final error also depends on optimization, random initialization, network capacity, and the derivative-computation method. In particular, AD and FDM are not minimizing exactly the same practical residual because FDM introduces finite-difference truncation and round-off effects through the choice of ϵ .

Part C

For this part, I studied how the network size affects the AD-PINN and FDM-PINN methods. I compared a smaller network with 2 hidden layers and 16 neurons per layer against a larger network with 5 hidden layers and 64 neurons per layer as the assignment suggested. For the ODE problem, I used $N_r = 500$ and trained for 10000 epochs. For the heat equation, I used $N_r = 10000$ and trained for 20000 epochs.

Method	Network	Layers	Neurons	Max Error	Time (s)
AD-PINN	Small	2	16	1.2576×10^{-2}	23.61
AD-PINN	Large	5	64	8.6949×10^{-3}	37.77
FDM-PINN	Small	2	16	1.2527×10^{-2}	26.23
FDM-PINN	Large	5	64	9.2264×10^{-3}	45.44

Above is the effect of network size on the ODE PINN results. For the ODE problem, the larger network improved the final max error for both AD-PINN and FDM-PINN. The AD-PINN error decreased from 1.2576×10^{-2} to 8.6949×10^{-3} , while the FDM-PINN error decreased from 1.2527×10^{-2} to 9.2264×10^{-3} . However, this improvement came with a clear increase in training time. The larger AD-PINN took 37.77 seconds compared to 23.61 seconds for the smaller AD-PINN, and the larger FDM-PINN took 45.44 seconds compared to 26.23 seconds for the smaller FDM-PINN.

Method	Network	Layers	Neurons	Relative L^2 Error	Time (s)
AD-PINN	Small	2	16	2.5470×10^{-3}	86.70
AD-PINN	Large	5	64	3.5131×10^{-3}	154.77
FDM-PINN	Small	2	16	2.5505×10^{-3}	105.94
FDM-PINN	Large	5	64	7.7791×10^{-3}	213.23

Above is the effect of network size on the heat-equation PINN results. For the heat equation, the larger network did not improve the final error in this run. The small AD-PINN and small FDM-PINN both achieved relative L^2 errors around 2.55×10^{-3} , while the large AD-PINN had a larger error of 3.5131×10^{-3} , and the large FDM-PINN had an even larger error of 7.7791×10^{-3} . This suggests that increasing network capacity does not automatically improve accuracy under a fixed training budget. A larger network has more parameters and can represent more complicated functions, but it is also harder to optimize and takes longer per epoch.

Overall, network capacity interacts with the derivative computation method through both accuracy and computational cost. For the ODE, the larger network helped both AD and FDM because the problem is relatively simple and the additional capacity was useful. For the heat equation, the larger network increased the training time substantially but did not improve the final error. The FDM-PINN was especially expensive for the large heat-equation network because each residual evaluation requires several network forward passes for the finite-difference stencil. In contrast, AD computes derivatives through automatic differentiation, which can be more direct but still becomes costly for PDEs involving second derivatives.

Part D

This project connects directly to the course ideas of convergence, rate of convergence, truncation error, and round-off error. In Problem 1.1, we used classical numerical methods, including Forward Euler and RK4, to solve the first-order ODE. The convergence table shows that decreasing the step size h decreases the global error, but the rate of decrease depends strongly on the method. Forward Euler has first-order convergence, so its global error decreases like $O(h)$. RK4 has fourth-order convergence, so its global error decreases like $O(h^4)$. This explains why RK4 produced a much smaller error than Forward Euler for the same step size. For the PINN methods, convergence has a different meaning. Instead

of refining a time step, the neural network is trained over many epochs so that the physics loss decreases. The loss curves show this training convergence. However, PINN convergence is not as predictable as the convergence rate of RK4 because the neural-network optimization problem is nonconvex and the collocation points are sampled randomly. Increasing the number of collocation points can also affect convergence because the loss becomes a better approximation of the continuous residual over the domain.

The project also shows the balance between truncation error and round-off error. In the FDM-PINN, derivatives are approximated using central-difference formulas with a parameter ϵ . If ϵ is too large, the derivative approximation is less accurate because the truncation error is larger. If ϵ is too small, the network values being subtracted become very close to each other, such as $u_\theta(t + \epsilon)$ and $u_\theta(t - \epsilon)$ in the ODE problem. Subtracting nearly equal floating-point numbers can cause cancellation and round-off error, and then dividing by ϵ can amplify that error. This explains why the FDM-PINN error improved as ϵ decreased at first, but became worse when ϵ was made too small. The same truncation-error and round-off-error tradeoff appears in the heat equation in Problem 2. There, the AD-PINN computes $\partial u_\theta / \partial t$ and $\partial^2 u_\theta / \partial x^2$ using automatic differentiation, while the FDM-PINN approximates these derivatives using central differences. The relative L^2 error on the 100×100 test grid measures how well the trained network approximates the exact heat-equation solution over the full space-time domain. The heat equation is especially sensitive to the choice of ϵ because the second-derivative approximation divides by ϵ^2 , which can amplify round-off error more strongly than the first-derivative approximation in the ODE problem. Thus, we show that accuracy depends not only on the method itself, but also on the rate of convergence, the size of the discretization or derivative parameter, and the balance between truncation and round-off error.

Bonus

For the bonus problem, as the assignment wanted, I treated the diffusion coefficient ν in the heat equation as an unknown trainable parameter. Instead of fixing

$$\nu = 0.01,$$

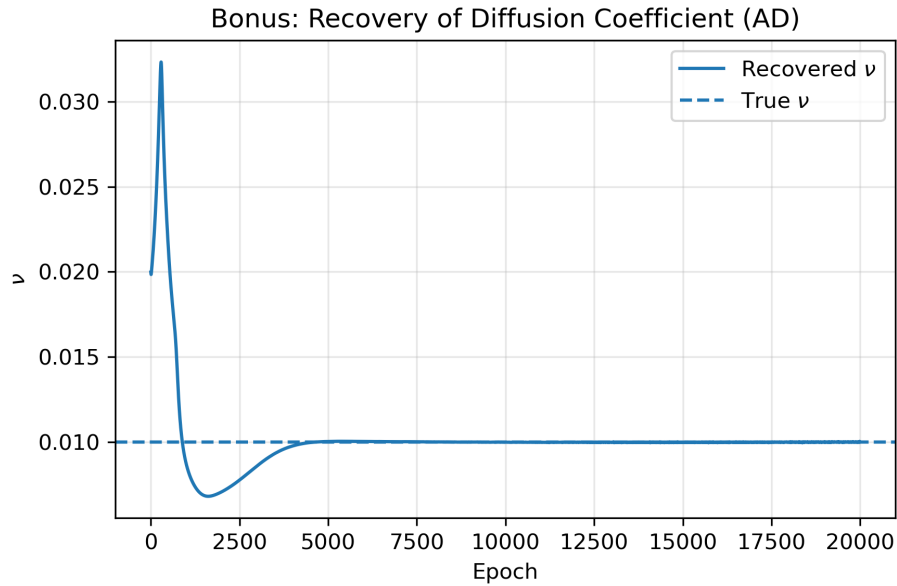
I trained the PINN to learn both the solution $u_\theta(x, t)$ and the parameter ν . To ensure that the learned value of ν stayed positive, I trained $\log(\nu)$ and then recovered ν by exponentiating it.

The inverse-problem loss included the PDE residual, the initial-condition loss, the boundary-condition loss, and a data-fitting loss from $N_{\text{data}} = 50$ noisy observations sampled from the exact solution with noise level $\sigma = 0.01$. The relative recovery error was computed as

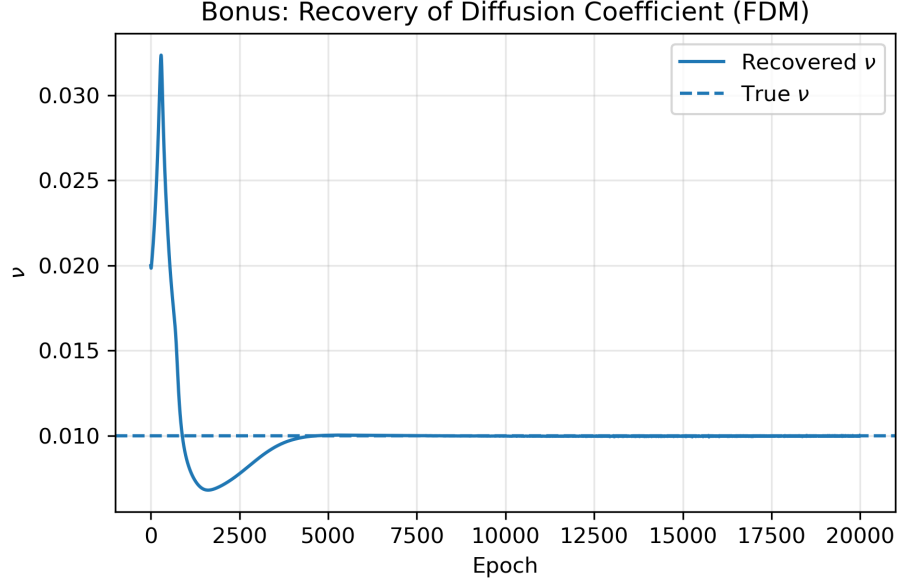
$$\frac{|\nu_{\text{recovered}} - \nu_{\text{true}}|}{|\nu_{\text{true}}|}.$$

Method	ν_{true}	$\nu_{\text{recovered}}$	Relative Error	Time (s)
AD-PINN	$1.00000000 \times 10^{-2}$	$1.00032901 \times 10^{-2}$	3.2901×10^{-4}	186.56
FDM-PINN	$1.00000000 \times 10^{-2}$	$1.00019025 \times 10^{-2}$	1.9025×10^{-4}	154.38

This is the recovered diffusion coefficient for the inverse heat-equation problem.



Above is the recovery of the diffusion coefficient ν using the AD-PINN inverse method.



Above is the recovery of the diffusion coefficient ν using the FDM-PINN inverse method. Both inverse PINN methods recovered the true diffusion coefficient very accurately. The AD-PINN recovered

$$\nu_{\text{AD}} = 1.00032901 \times 10^{-2},$$

giving a relative error of

$$3.2901 \times 10^{-4}.$$

The FDM-PINN recovered

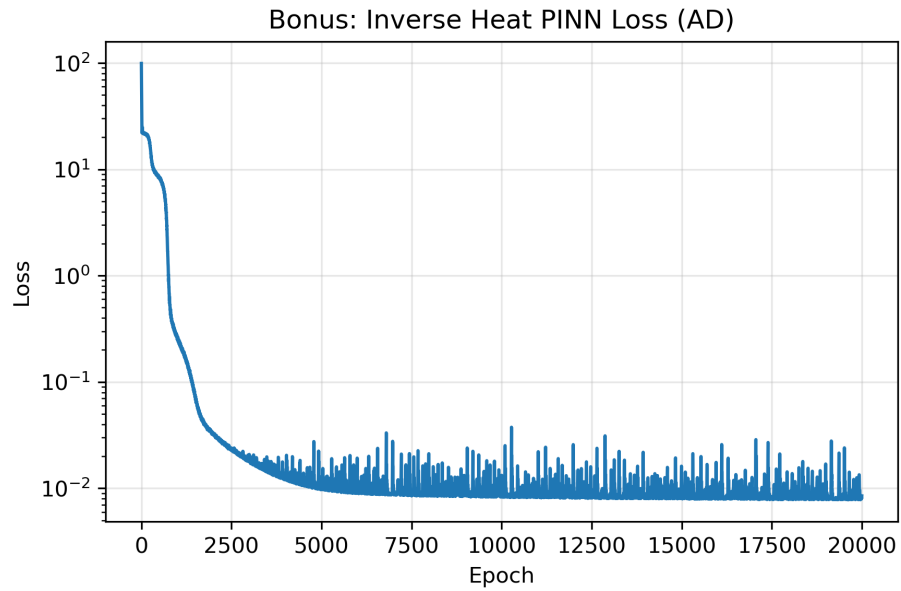
$$\nu_{\text{FDM}} = 1.00019025 \times 10^{-2},$$

giving a relative error of

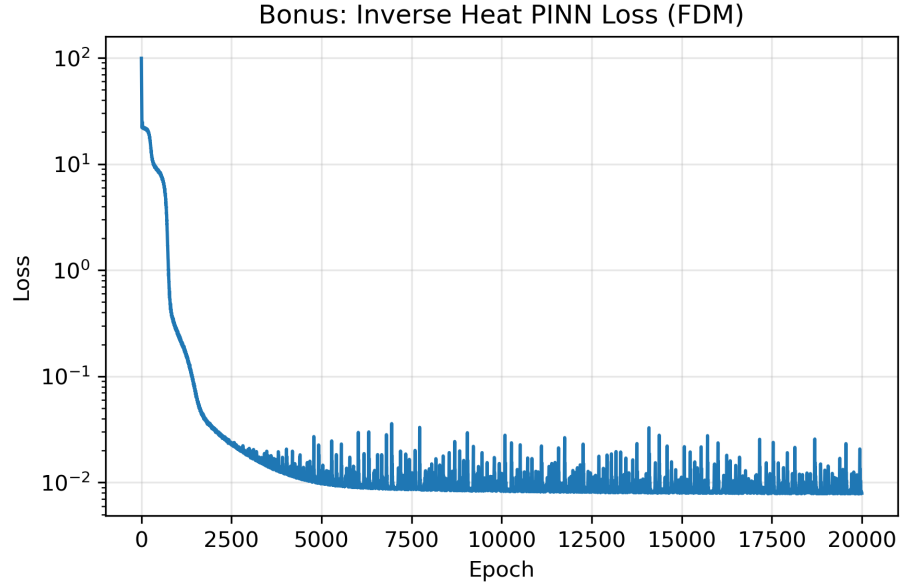
$$1.9025 \times 10^{-4}.$$

In this run, the FDM-PINN recovered ν slightly more accurately and also trained faster. However, both methods converged very close to the true value $\nu_{\text{true}} = 0.01$, so both successfully solved the inverse problem.

The recovery plots show that the learned value of ν initially moves away from the true value, but then returns and stabilizes near $\nu = 0.01$. This is reasonable because the network is simultaneously learning the solution $u_\theta(x, t)$ and the unknown coefficient ν . Early in training, the predicted solution is still inaccurate, so the parameter estimate can fluctuate. Once the solution and residual losses become smaller, the learned diffusion coefficient stabilizes near the true value. Below is the training loss for the AD-PINN inverse heat-equation problem.



below is the training loss for the FDM-PINN inverse heat-equation problem.



The loss plots show that both inverse PINN methods have the same general training pattern. At the beginning of training, the loss is very large because the network has not yet learned the heat-equation solution or the unknown

parameter ν . During the first few thousand epochs, the loss decreases rapidly by several orders of magnitude. After this initial stage, the loss levels off near 10^{-2} , with occasional spikes caused by the random resampling of collocation points during training. Since each epoch uses newly sampled residual, initial-condition, and boundary-condition points, the loss is not expected to decrease perfectly smoothly.

In this run, the FDM-PINN recovered ν slightly more accurately and also trained faster, taking 154.38 seconds compared with 186.56 seconds for the AD-PINN. However, both methods successfully identified the diffusion coefficient, with recovered values very close to the true value $\nu_{\text{true}} = 0.01$.

The ν -recovery plots show an important feature of the inverse problem. The learned parameter initially overshoots and undershoots the true value before stabilizing near $\nu = 0.01$. This is reasonable because the network is learning both the function $u_\theta(x, t)$ and the coefficient ν at the same time. Early in training, the predicted solution is still inaccurate, so the optimizer may temporarily adjust ν in the wrong direction to reduce the combined loss. Once the solution approximation improves, the parameter estimate becomes much more stable and converges close to the true diffusion coefficient.